Cahier des charges — Moteur de recherche HeRVé

S2.02 · Exploration algorithmique d'un problème · UniCA 2025-2026

Avril 2026 · v1.0

1. Synthèse exécutive

> **Mission.** Concevoir et livrer **HeRVé**, un moteur de recherche en Java capable d'indexer un site web francophone de taille moyenne (10 000 – 50 000 pages) et de répondre à une requête en langage naturel en **moins de 200 ms**, avec un classement par pertinence.

Élément	Valeur
Nom de code	HeRVé
Domaine	Recherche d'information (Information Retrieval)
Modèle	Vector Space Model — sac de mots + TF-IDF + cosinus
Corpus cible	`fr.vikidia.org` — *encyclopédie francophone pour enfants* (~38 000 articles)
Stack	Java 17 (OpenJDK), aucune dépendance externe
Plateforme cible	Linux Debian 12
Échéance finale	**vendredi 12 juin 2026, 20 h 00**
Soutenance	Semaine 25 — démonstration + répartition individuelle des points

Pourquoi ce projet est intéressant. Construire un moteur de recherche oblige à toucher quasiment toutes les briques fondamentales d'un système : I/O fichiers, structures de données performantes, traitement du langage naturel (NLP), algèbre linéaire, performance, tests, et — pour la dernière itération — interface utilisateur et architecture distribuée. C'est un terrain d'apprentissage exceptionnellement complet.

2. Contexte & vision produit

2.1 Le problème à résoudre

Un utilisateur final tape une suite de mots dans une barre de recherche. Il s'attend à ce que :

1. **les bonnes pages remontent en premier** (pertinence) ;
2. **la réponse soit instantanée** (< 200 ms perçues) ;
3. **les fautes de frappe et variations morphologiques** (pluriels, conjugaisons) ne pénalisent pas la recherche ;
4. **l'interface soit aussi évidente qu'une barre Google** — un champ, un bouton, dix résultats.

2.2 Vision long terme

> *« Un moteur de recherche qui comprend l'intention, pas seulement les mots. »*

À l'horizon de la SAÉ, on ne va pas jusqu'à la compréhension sémantique. On vise un **socle vectoriel solide**, extensible, sur lequel viennent se brancher : lemmatisation, filtrage de mots vides, TF-IDF, autocorrection lexicale, recherche littérale entre guillemets, et indexation à froid d'un site web arbitraire.

2.3 Principes directeurs (engineering tenets)

1. **Simplicité d'abord, optimisation ensuite.** On code le modèle minimal qui marche, on mesure, on optimise.
2. **Mesurer avant d'optimiser.** Aucune optimisation n'est admise sans benchmark à l'appui.
3. **Tester à chaque étape.** Tout code public est couvert par un test au plus tard à la même version.
4. **Stabilité des interfaces.** Les signatures publiques (classes, scripts d'entrée) sont gelées dès qu'elles sont validées — l'optimisation se fait *derrière* l'interface.
5. **Frugalité.** Aucune dépendance externe : tout passe par la JDK standard.

---

3. Personas & user stories

3.1 Personas

Persona 1 — Léa, 11 ans, élève de 6e

> *« Je cherche un truc pour mon exposé sur le Royaume-Uni. »*

- **Comportement** : tape 2 à 5 mots, fautes d'orthographe fréquentes, ne lit que les 3 premiers résultats.
- **Besoin** : que le 1er résultat soit pertinent même quand elle écrit *« roiyaume uni »*.

Persona 2 — M. Dupuis, professeur des écoles

> *« Je veux préparer un cours sur la photosynthèse. »*

- **Comportement** : requêtes longues (5–10 mots), formulation propre, exigeant sur la pertinence.
- **Besoin** : pouvoir mettre une expression entre guillemets pour forcer une co-occurrence (`"chaîne alimentaire"`).

Persona 3 — Le développeur évaluateur (l'enseignant)

> *« Je dois pouvoir noter 12 projets en un soir. »*

- **Comportement** : exécute des scripts d'évaluation automatisés.
- **Besoin** : une interface CLI **stable, prévisible et conforme** au `README.md` du dépôt — `compile_projet` puis `herve`.

3.2 User stories prioritaires

ID	Story	Priorité	Version cible
US-01	En tant qu'utilisateur, je tape une requête en CLI et obtiens les 15 résultats les plus pertinents triés.	P0	v0.2
US-02	En tant qu'utilisateur, je peux enchaîner plusieurs recherches sans relancer le programme.	P0	v0.2
US-03	En tant qu'utilisateur, je tape « fruits » et obtiens les pages contenant « fruit ».	P0	v0.3
US-04	En tant qu'utilisateur, j'obtiens une réponse en moins de 200 ms sur 38 000 pages.	P0	v0.4
US-05	En tant qu'utilisateur, j'utilise le moteur via mon navigateur.	P0	v0.5
US-06	En tant qu'utilisateur, je tape `"pommes de terre"` entre guillemets pour forcer l'expression exacte.	P1	Final
US-07	En tant qu'utilisateur, je tape une faute et le moteur la corrige.	P2	Final
US-08	En tant qu'opérateur, je peux indexer à froid un site web arbitraire.	P2	Final

4. Architecture & modèle algorithmique

4.1 Vue d'ensemble

Deux pipelines distincts qui ne partagent que **le format des fichiers d'index** :

```
[Indexation à froid (offline)]                    [Recherche à chaud (online)]

  page web HTML                                      requête utilisateur
        |                                                   |
        ▼                                                   ▼
  extraction texte                                    tokenisation
        |                                                   |
        ▼                                                   ▼
  tokenisation                                        lemmatisation
        |                                                   |
        ▼                                                   ▼
  lemmatisation                                       filtrage mots vides
        |                                                   |
        ▼                                                   ▼
  comptage occurrences                                vecteur requête r
        |                                                   |
        ▼                                                   ▼
  fichier d'index ──────────────────────────▶ chargement pages indexées
                                                            |
                                                            ▼
                                                      calcul similarité
                                                            |
                                                            ▼
                                                      tri & top-15
```

4.2 Modèle vectoriel (sac de mots)

Soit $V = \{w_1, w_2, \ldots, w_n\}$ le **vocabulaire** : l'union des mots (ou lemmes) présents dans au moins un document du corpus. Tout document d — comme toute requête r — est représenté par un vecteur $\vec{d} \in \mathbb{R}^n$ dont la i-ième coordonnée est le **nombre d'occurrences** du mot w_i dans d :

$$\vec{d} = (\mathrm{tf}_{1,d}, \mathrm{tf}_{2,d}, \ldots, \mathrm{tf}_{n,d})$$

L'ordre des mots dans le document est ignoré ; on ne garde que la **multiplicité**. D'où le terme *bag of words*.

4.3 Mesure de similarité : cosinus

La pertinence d'un document d pour une requête r est mesurée par le **cosinus de l'angle** entre les deux vecteurs :

$$\mathrm{sim}(\vec{r}, \vec{d}) = \cos(\theta) = \frac{\vec{r} \cdot \vec{d}}{\|\vec{r}\| \cdot \|\vec{d}\|} = \frac{\sum_{i=1}^{n} r_i \, d_i}{\sqrt{\sum_{i=1}^{n} r_i^2} \cdot \sqrt{\sum_{i=1}^{n} d_i^2}}$$

Pourquoi le cosinus plutôt que le simple produit scalaire ? Parce qu'il est **invariant par homothétie** : un long document contenant les mots de la requête en abondance n'est pas mécaniquement avantagé. La similarité ne dépend que de la **direction** des vecteurs, pas de leur norme.

Propriétés :

- $\cos(\theta) = 0$: les deux vecteurs sont **orthogonaux** — *aucun mot en commun* entre la requête et le document.
- $\cos(\theta) = 1$: les deux vecteurs sont **colinéaires** — *même profil de mots*, à la fréquence relative près.

4.4 Pondération TF-IDF

Tous les mots ne se valent pas. *« cuisine »* discrimine moins que *« paella »* sur un corpus de recettes. On pondère donc la i-ième coordonnée par un **coefficient de pertinence** :

$$c_i = \log\left(\frac{D}{d_i}\right)$$

où D est le nombre total de documents et d_i le nombre de documents contenant le mot w_i. Le vecteur final d'un document est :

$$\vec{d}^{\,\mathrm{tfidf}} = (c_1 \cdot \mathrm{tf}_{1,d},\ c_2 \cdot \mathrm{tf}_{2,d},\ \ldots,\ c_n \cdot \mathrm{tf}_{n,d})$$

Lecture intuitive : un mot présent dans tous les documents a $d_i = D$, donc $c_i = \log(1) = 0$ — il ne pèse plus rien. Un mot rare, présent dans 1% des pages, a $c_i = \log(100) \approx 4{,}6$ — il devient déterminant.

4.5 Lemmatisation

Le moteur travaille en *lemmes*, pas en *mots de surface*. Le lemme est la forme canonique :

Surface	Lemme
mangeront, mangeais, mangées	manger
beau, belle, beaux, belles	beau
pommes	pomme

L'algorithme adopté est **dictionnaire-based** : un fichier `french_dictionary.txt` (~336 000 entrées, format `mot:lemme`) est consulté en O(1) via une `HashMap<String, String>`.

Limitation acceptée : pas d'analyse grammaticale. Les homonymes (*« est »* → être ou point cardinal ?) ne sont pas désambiguïsés ; l'erreur est statistiquement marginale et corrigée au mieux par le contexte vectoriel global.

4.6 Filtrage des mots vides (stopwords)

Une *blacklist* (~50 mots français : *le, la, de, à, et, …*) élimine les termes hyper-fréquents qui n'apportent aucun signal sémantique. Ce filtrage s'applique :

1. **À l'indexation** (fait en amont, déjà appliqué dans les fichiers fournis) ;
2. **À la requête** (à notre charge — sinon `"pomme de terre"` voit `de` matcher partout).

4.7 Autocorrection (rendu final)

Pour gérer les fautes de frappe, on utilise une **distance d'édition**. La distance de Levenshtein $L(a, b)$ est définie par récurrence :

$$L(a,b) = \begin{cases} \max(|a|,|b|) & \text{si } \min(|a|,|b|) = 0 \\ \min \begin{cases} L(a-1,b)+1 \\ L(a,b-1)+1 \\ L(a-1,b-1)+\mathbb{1}_{a_i \neq b_j} \end{cases} & \text{sinon} \end{cases}$$

Coût : $O(|a| \cdot |b|)$ par paire — prohibitif sur un dictionnaire de 100 k mots. **Solution en deux passes** :

1. **Pré-sélection rapide** par distance de Jaccard sur les n-grammes :
$$J(A,B) = \frac{|A \cap B|}{|A \cup B|}$$
2. **Affinage** par Levenshtein sur les ~50 candidats retenus.

4.8 Recherche littérale (rendu final)

Pour la requête `"pommes de terre" gâteau` :

1. **Phase 1 — vectorielle** : tri des pages par cosinus avec = vecteur de `{pomme, terre, gâteau}` (lemmatisé, sans stopwords).
2. **Phase 2 — filtrage exact** : sur les *k* meilleures pages, recherche de la chaîne `pommes de terre` dans le **contenu brut** (archive des textes complets, fournie séparément, 17 Mo).
3. **Re-classement** : seules les pages passant la phase 2 sont retournées.

---

5. Roadmap par version

5.1 v0.1 — Structure de base · *échéance 15 mai 2026*

Objectif : prototype en mode test pur (aucun I/O fichier, aucune CLI). On valide le **cœur algorithmique**.

Périmètre fonctionnel

- Représentation d'une page comme un vecteur de mots/occurrences.
- Calcul de la norme et de la pondération.
- Calcul de la similarité entre deux pages (produit scalaire normalisé).

Architecture

Package `search_engine` :- `IndexedPage` - `SearchEngine` *(squelette)* - `SearchResult` *(squelette)*

Package `search_engine_tests` :- `SearchEngineTests`

Spécification de `IndexedPage`

Attributs (privés)

```
private String url;
private String[] words;
private int[] counts;
```

Constructeurs

- `IndexedPage(String[] lines)` — construit une page depuis un tableau de lignes : la 1re ligne est l'URL, les suivantes sont au format `mot:occurrences`.
- `IndexedPage(String text)` — construit une page « requête » à partir d'une chaîne libre : split sur les espaces, comptage des doublons, tri lexicographique des mots.

> ⚠ **Hors périmètre v0.1** : `IndexedPage(Path path)` — reporté en v0.2.

Méthodes publiques

Signature	Sémantique
`String getUrl()`	retourne l'URL
`int getCount(String word)`	nombre d'occurrences du mot dans la page
`double getNorm()`	norme euclidienne du vecteur d'occurrences
`double getPonderation(String word)`	$\frac{\mathrm{tf}_w}{\lVert\vec{d}\rVert}$ — coordonnée normalisée du mot
`double proximity(IndexedPage page)`	produit scalaire normalisé entre les deux pages
`String toString()`	`"IndexedPage [url=...]"`

Test d'acceptation

```
IndexedPage page1 = new IndexedPage(new String[] {
    "http://fr.example.org", "hello:10", "world:5"
});
System.out.println(page1);
// → IndexedFile [url=http://fr.example.org]
System.out.println(page1.getPonderation("hello"));
// → 0.8944271909999159
System.out.println(page1.getPonderation("other"));
// → 0.0

IndexedPage page2 = new IndexedPage(new String[] {
    "http://fr.example2.org", "hello:5", "..."
});
System.out.println(page1.proximity(page2));
// → 0.8304547985373998
```

Critères d'acceptation v0.1

- ☐Branche `v0.1` créée, tag `v0.1` posé sur le commit final.
- ☐Scripts `compile_projet` et `herve` toujours fonctionnels.
- ☐ `SearchEngineTests` couvre **les deux constructeurs** et **chaque méthode publique** d' `IndexedPage`.
- ☐Sortie attendue obtenue à 10^{-10} près sur l'exemple de référence.

5.2 v0.2 — Entrées/Sorties · *échéance 22 mai 2026*

Objectif : moteur fonctionnel sur un **corpus de 100 pages** Vikidia (gastronomie), pilotable en CLI.

Périmètre fonctionnel

1. Lecture d'un fichier d'index depuis le disque.
2. Lecture récursive d'un dossier d'index.
3. Lecture d'une requête depuis la ligne de commande.
4. Calcul des similarités, tri, **affichage du Top-15**.
5. **Deux modes d'exécution** :
   - **One-shot** : `java -cp . search_engine.SearchEngine flan cerise` → résultats puis terminaison.
   - **Interactif** : `java -cp . search_engine.SearchEngine` → boucle prompt → résultat → prompt → … → `exit`.

Format des fichiers d'index

```
https://fr.vikidia.org/wiki/Tarte
pomme:14
pâte:9
sucre:7
...
```

- **Encodage** : UTF-8.
- **Fin de ligne** : `\n` (UNIX).
- **Ligne 1** : URL canonique.
- **Lignes 2..n** : `<lemme>:<entier>` — exactement un `:`, pas d'espace.

Spécification de `SearchResult`

```
public class SearchResult {
    private String url;
    private double score;

    public SearchResult(String url, double score) { ... }
    public String getUrl() { ... }
    public double getScore() { ... }
    public String toString() { ... }   // "SearchResult [url=..., score=...]"
}
```

Spécification de `SearchEngine`

Attributs

```
private Path indexation_directory;
private IndexedPage[] pages;
```

Constructeur

`SearchEngine(Path indexation_directory) throws IOException` - Liste tous les fichiers du dossier. - Construit un `IndexedPage` par fichier. - Stocke le tableau dans `pages`.

Méthodes publiques

Signature	Sémantique
`IndexedPage getPage(int i)`	accès direct à la i-ième page
`int getPagesNumber()`	cardinal du corpus indexé
`SearchResult[] launchRequest(String requestString)`	exécute la requête, renvoie tous les résultats triés
`void printResults(String requestString)`	affiche le **Top-15**, scores nuls exclus

Méthode `main` — point d'entrée

- Si `args.length > 0` : concaténer `args` en requête, appeler `printResults`, terminer.
- Sinon : boucle `Scanner` jusqu'à saisie de `exit`.

Résolution du chemin INDEX (recommandation)

```
URL location = SearchEngine.class.getProtectionDomain()
                                  .getCodeSource()
                                  .getLocation();
Path binFolder = Paths.get(location.toURI());
Path indexFolder = binFolder.resolve("INDEX");
```

Cette approche fonctionne **à la fois** dans Eclipse et en exécution CLI, contrairement à `Paths.get("INDEX")` qui dépend du `cwd`.

Critères d'acceptation v0.2

- ☐ `IndexedPage(Path path) throws IOException` implémenté.
- ☐ Résultat de référence obtenu sur la requête `cerise flan` :

```
SearchResult [url=https://fr.vikidia.org/wiki/Flan,        score=37.71273413657018]
SearchResult [url=https://fr.vikidia.org/wiki/Clafoutis,   score=16.116459280507602]
SearchResult [url=https://fr.vikidia.org/wiki/Tarte,       score=5.076730825668095]
```

- ☐ Tests unitaires sur **toutes** les méthodes publiques de `SearchEngine` et `SearchResult`.

- ☐Branche `v0.2`, tag `v0.2`.

5.3 v0.3 — Lemmatisation · *échéance 26 mai 2026*

Objectif : aligner la requête sur l'index — qui est déjà lemmatisé. Sans cette version, taper « *fruits* » renvoie 0 résultat alors que l'index contient « *fruit* ».

> 🔔 **Inflexion pédagogique.** À partir d'ici, *vous n'êtes plus guidés*. Les choix d'architecture deviennent **votre responsabilité**. En soutenance, vous devrez les **expliquer** et les **défendre** : *quelles alternatives ? pourquoi celle-ci ? quels benchmarks ?*

Périmètre fonctionnel

1. **Conversion en minuscules** de la requête.
2. **Lemmatisation** mot à mot via le dictionnaire fourni.
3. **Filtrage des mots vides** via une blacklist fournie (modifiable).
4. **Robustesse aux caractères non-alphabétiques** (ponctuation, chiffres, accents).

Ressources fournies

Fichier	Taille	Format	Usage
`french_dictionary.txt`	7,4 Mo	UTF-8, `mot:lemme` par ligne	Lemmatisation
`blacklist.txt`	217 o	UTF-8, un mot par ligne	Filtrage stopwords

> 🚫 **Ne pas modifier le dictionnaire** : la lemmatisation appliquée à la requête doit rester identique à celle appliquée à l'index. Toute amélioration nécessiterait de **régénérer l'index**, ce qui n'est pas l'objet de cette version.

Décisions à prendre (et à défendre)

- **Découpage du texte** : split par regex `[^\p{L}]+` ? Tokenizer custom ?
- **Mots courts** : on filtre les mots de 1 ou 2 lettres ? Seuil ?
- **Mots inconnus du dictionnaire** : on les garde (forme de surface) ou on les ignore ?
- **Casse des accents** : « *été* » et « *ETE* » doivent-ils matcher ? (oui — `String.toLowerCase(Locale.FRANCE)`)

Critères d'acceptation v0.3

- ☐La requête `"les fruits"` retourne les mêmes résultats que `"fruit"`.
- ☐La requête `"pommes ! de ?? terre."` est interprétée comme `"pomme terre"`.

- ☐Aucun stopword ne pollue le calcul de similarité.
- ☐Tests unitaires sur les nouveaux modules de prétraitement.
- ☐Branche `v0.3`, tag `v0.3`.

5.4 v0.4 — Passage à l'échelle · *échéance 29 mai 2026*

Objectif : passer de 100 à **38 000 pages** en gardant un temps de réponse acceptable.

Le problème de fond

Sur 38 000 pages × ~1 000 mots distincts par page, la naïveté algorithmique se paye au prix fort. Trois axes d'attaque :

1. **Structures de données.** `String[] words` + `int[] counts` impose une recherche *O(n)* par mot. Une `HashMap<String, Integer>` ramène cela à *O(1)* amorti.
2. **Pré-calcul.** La norme `||d||` est constante — elle peut être calculée une fois à l'indexation et stockée dans `IndexedPage`.
3. **Lazy I/O.** Pas besoin de tout charger en RAM si l'on streame intelligemment.

Recommandations méthodologiques

1. **Dérouler en pyramide** : 100 → 1 000 → 10 000 → 38 000. Chaque palier doit rester sous le seuil de réponse cible.
2. **Benchmark systématique** : mesurer (`System.nanoTime()`) le temps d'indexation initial *et* le temps moyen par requête, à chaque palier, pour chaque structure testée.
3. **Stabilité de l'API** : seules les structures *internes* changent. Les signatures publiques d'`IndexedPage` et `SearchEngine` sont gelées depuis v0.2.

Tableau de benchmark à produire pour la soutenance

Corpus	Structure	Temps indexation	Temps moyen / requête	RAM résidente
100 pages	`String[]`	… ms	… ms	… Mo
100 pages	`HashMap`	… ms	… ms	… Mo
1 000 pages	`String[]`	… ms	… ms	… Mo
1 000 pages	`HashMap`	… ms	… ms	… Mo
38 000 pages	`HashMap`	… ms	… ms	… Mo

Critères d'acceptation v0.4

- ☐ Le moteur charge les **38 000 pages** sans `OutOfMemoryError` sur une JVM par défaut.
- ☐ Temps moyen par requête ≤ **200 ms** sur 38 000 pages (objectif KPI).
- ☐ Au moins **deux structures de données** comparées avec benchmarks chiffrés.
- ☐ Branche `v0.4`, tag `v0.4`.

Données fournies

- `INDEX_FILES.7z` (6,3 Mo compressé) — fichiers d'index complets de `fr.vikidia.org`.

5.5 v0.5 — Interface web · *échéance 12 juin 2026*

Objectif : exposer le moteur via un navigateur. Une barre, un bouton, des résultats.

Périmètre fonctionnel

1. Serveur HTTP minimal (`com.sun.net.httpserver.HttpServer` — *inclus dans la JDK, pas de dépendance*).
2. Endpoint `GET /search?q=...` retournant les résultats au format JSON ou HTML.
3. Page d'accueil servie statiquement.
4. Conformité RGAA basique (champ texte labellisé, navigation clavier).

Architecture proposée

```
Navigateur ── HTTP GET /search?q=... ──▶ HttpServer Java
                                              │
                                              ▼
                                         SearchEngine.launchRequest()
                                              │
                                              ▼
                                         JSON serialization
                                              │
                                   HTTP 200 ── │
                                              ▼
                                         rendu DOM côté client
```

Spécification d'API

Méthode	Path	Paramètres	Réponse
GET	`/`	—	HTML : page d'accueil avec champ de recherche
GET	`/search`	`q=<string>`	`200 OK` JSON `[{"url":"...","score":0.83},...]`
GET	`/search`	(pas de `q`)	`400 Bad Request`

Décisions techniques à acter

- **Sérialisation JSON** : à la main (concaténation `String`) — pas de Jackson/Gson, contrainte « zéro dépendance ».
- **Encodage** : UTF-8 partout, header `Content-Type: application/json; charset=utf-8`.
- **Cache** : `Cache-Control: no-store` pour le développement.
- **Concurrence** : un thread par requête suffit pour la démo (`HttpServer.create(...).setExecutor(Executors.newFixedThreadPool(4))`).

Critères d'acceptation v0.5

- ☐ Le serveur démarre via `./herve serve` (à intégrer au script d'entrée).
- ☐ La page d'accueil s'affiche sur `http://localhost:8080`.
- ☐ Une recherche complète boucle en moins de **300 ms** côté serveur (target wall-clock).
- ☐ Branche `v0.5`, tag `v0.5`.

5.6 Rendu final — *Soyez créatifs ! · échéance 12 juin 2026*

> *« Vous avez le droit à l'échec, ce n'est pas dramatique. »* — l'enseignant

Il faut choisir **une** piste d'amélioration et la **mener à bien** (ou documenter pourquoi elle a échoué). Trois pistes officielles, par ordre de difficulté croissante :

Piste A — Indexation à froid (cold indexing)

> Construire son propre index, pour pouvoir indexer **n'importe quel site**, pas seulement Vikidia.

Sous-tâches :

1. **HTTP client** : `HttpClient.newHttpClient()` (JDK 11+). Une requête GET, parsing du body.

2. **Parsing HTML** : *jericho*, *jsoup*… interdits. Solution acceptable : `String.replaceAll("<[^>]+>", "")` + nettoyage des entités HTML les plus courantes (`&` , ` ` , …).
3. **Lemmatisation locale** : réutiliser le `french_dictionary.txt` .
4. **Crawler** : extraction des liens via regex `href="([^"]+)"` , file d'attente, visited-set, **politesse** (1 seconde entre deux requêtes même hôte).

> ⚠️ **Politesse réseau obligatoire.** L'IP qui crawl trop vite se fait bannir — c'est de l'apprentissage par la douleur qu'on préfère éviter.

Piste B — Autocorrection

Pré-sélection Jaccard sur n-grammes (n = 2 ou 3) → top 50 candidats → re-classement Levenshtein → suggestion la plus proche si distance ≤ 2.

Piste C — Améliorer la pertinence

Recherche littérale entre guillemets (cf. §4.8) ou évaluation de la **distance entre les mots** dans la page.

Données complémentaires fournies

- Archive `contenus_textuels_bruts.7z` (17 Mo) : texte brut de chaque page, **même nom de fichier que l'index** → jointure triviale par nom.

Pour toutes les pistes

- **Documenter** la démarche, les choix, les résultats.
- **Évaluer** la pertinence avec des exemples.
- **Faire preuve d'esprit critique** — c'est noté autant que la réussite technique.

---

6. Spécifications techniques transverses

6.1 Stack imposée

Élément	Choix	Justification
Langage	**Java 17 (OpenJDK)**	imposé par l'énoncé
Dépendances	**aucune** hors JDK	imposé par l'énoncé
OS d'évaluation	**Debian 12**	imposé
IDE recommandé	Eclipse	imposé (autorisé : autres IDE, *si* arborescence `src/` / `bin/` respectée)
Build	scripts shell maison `compile_projet`	imposé — **ne pas modifier**

6.2 Arborescence du dépôt

```
.
├── src/                            ← sources Java
│   └── search_engine/
│   └── search_engine_tests/
├── bin/                            ← classes compilées (regénéré)
│   └── INDEX/                      ← fichiers d'index (copie depuis ressources/)
├── doc/                            ← documentation
├── ressources/                     ← données brutes (dico, blacklist, archives)
├── compile_projet                  ← script — NE PAS MODIFIER
├── herve                           ← script — NE PAS MODIFIER
└── README.md                       ← interface CLI exacte
```

6.3 Workflow Git · GitLab IUT

- **Hébergeur** : `https://iut-git.unice.fr/s201-25-26/<groupe>/`
- **Procédure** :
  1. Fork du dépôt template `https://iut-git.unice.fr/s201-25-26/herve` dans le groupe (PAS dans un sous-groupe — l'enseignant exécute un script qui ne descend pas).
  2. **Ne pas renommer** le projet forké.
  3. Le chef de projet doit avoir le rôle **Maintainer** (pour pouvoir supprimer / régler les permissions).
- **Branches & tags par version** :

```
git checkout -b v0.X
# ... travail ...
git commit -am "v0.X complète"
git tag v0.X -m "version 0.X"
git push --tags origin v0.X
```

- **Discipline du nommage** : un tag mal nommé = un rendu non détecté = un zéro.

6.4 Connexion VPN

L'accès SSH au GitLab IUT passe par le **VPN universitaire** depuis l'extérieur du réseau IUT.

---

7. KPIs & métriques de succès

7.1 KPIs produit (mesurables)

Métrique	Cible v0.4	Cible finale	Méthode de mesure
Temps moyen par requête (P50)	≤ 200 ms	≤ 100 ms	benchmark JMH ou `nanoTime()` × 1 000 requêtes
Temps d'indexation initial	≤ 60 s	≤ 30 s	timer au démarrage
Empreinte mémoire	≤ 1 Go	≤ 512 Mo	`Runtime.totalMemory() - freeMemory()`
Top-1 pertinence subjective	≥ 70 %	≥ 85 %	échantillon de 30 requêtes étalon évaluées à la main
Couverture des tests	≥ 70 % des méthodes publiques	≥ 90 %	revue manuelle

7.2 KPIs projet

Métrique	Cible
Versions livrées dans les délais	6/6
Tags Git correctement posés	6/6
Scripts `compile_projet` / `herve` toujours fonctionnels	100 %
Documentation à jour à chaque version	oui

---

8. Risques & mitigations

Risque	Probabilité	Impact	Mitigation
Tag Git mal nommé → rendu non détecté	Moyenne	**Critique**	check-list pre-push, `git tag -l` avant chaque livraison
`OutOfMemoryError` sur 38 000 pages	Forte	Élevé	profilage dès 1 000 pages, refactor `String[]` → `HashMap`
Modification accidentelle de `compile_projet` ou `herve`	Faible	Critique	`.gitattributes` + revue de PR systématique sur ces fichiers
Lemmatisation incohérente entre index et requête	Moyenne	Élevé	utiliser le **même** dictionnaire — interdiction d'en générer un nouveau
Crawler banni (IP blacklistée) en piste A	Forte	Moyen	délai 1 s + `User-Agent` explicite + respect `robots.txt`
Conflits Git en équipe	Forte	Moyen	branches courtes, PR / MR avant merge, revue par le chef de projet
Membre absent / non-contributeur	Moyenne	Élevé	individualisation des notes en fin de projet (chef de projet)

---

9. Organisation, équipe & gouvernance

9.1 Constitution des équipes

TD	Effectif	Composition
G1	14	2 × 5 + 1 × 4
G2	17	2 × 6 + 1 × 5
G3	15	3 × 5
G4	17	2 × 6 + 1 × 5

Échéance constitution : **30 avril 2026 — 20 h 00**. À défaut, l'enseignant compose les équipes d'office.

9.2 Rôle du chef de projet

- **Pas un super-contributeur**, un **coordinateur**.
- Responsabilités : gestion du dépôt, répartition des tâches, suivi des échéances, lien avec l'enseignant, **répartition finale des points** (en concertation avec l'équipe).
- Doit avoir le rôle **Maintainer** sur GitLab.
- Une absence de chef compétent = effondrement de l'équipe : **choisir avec soin**.

10. Glossaire

Terme	Définition
Bag of words	modèle représentant un document comme un multiset de mots, sans ordre
Cosinus de similarité	$\cos(\theta) = \frac{\vec{u}\cdot\vec{v}}{\lVert\vec{u}\rVert\lVert\vec{v}\rVert}$ — voir §4.3
Crawler	programme qui parcourt automatiquement les liens d'un site
IDF	*Inverse Document Frequency* — voir §4.4
Indexation	phase off-line de construction de l'index inversé
Jaccard (distance de)	$1 - \frac{\lvert A\cap B\rvert}{\lvert A\cup B\rvert}$
Lemme	forme canonique d'un mot (infinitif, masculin singulier, ...)
Levenshtein (distance de)	nombre minimal d'éditions (insert, delete, substitute) pour transformer une chaîne en une autre
Stopword	mot vide, hyper-fréquent et non-discriminant
TF	*Term Frequency* — nombre d'occurrences d'un terme dans un document
TF-IDF	pondération TF × IDF — voir §4.4
Vikidia	encyclopédie collaborative francophone destinée aux 8-13 ans

11. Annexes

11.1 Échéancier consolidé

Semaine	Heures	Livrable
S18	2 h	Constitution équipes + TP Python
S19	2 h	Mise en place Git + démarrage v0.1
S20	2 h	(suite v0.1) — **livraison v0.1 le 15/05**
S21	10 h	v0.2 → v0.3 — **livraisons 22/05 et 26/05**
S22	8 h	v0.4 — **livraison 29/05**
S23	8 h	v0.5 — **livraison 12/06**
S24	8 h	Version finale — **livraison 12/06**
S25	4 h	Démo + soutenance + répartition des points

11.2 Ressources fournies (récapitulatif)

Fichier	Version cible	Usage
`INDEX.zip`	v0.2	corpus mini (100 pages)
`french_dictionary.txt`	v0.3	dictionnaire de lemmatisation
`blacklist.txt`	v0.3	stopwords français
`INDEX_FILES.7z`	v0.4	corpus complet (~38 000 pages)
`contenus_textuels_bruts.7z`	Final	textes bruts pour recherche littérale

11.3 Commandes de référence

```
# Compilation
./compile_projet

# Recherche one-shot
./herve flan cerise

# Recherche interactive
./herve

# Démarrage serveur web (v0.5)
./herve serve

# Indexation à froid d'un site (rendu final, optionnel)
./herve index https://exemple.org
```

11.4 Pour aller plus loin

- *Manning, Raghavan, Schütze — Introduction to Information Retrieval, Cambridge 2008* (gratuit en ligne).
- BM25 (Okapi) — successeur de TF-IDF, hors-scope mais **excellent sujet de présentation orale**.
- PageRank — orthogonal au modèle vectoriel, combinable.

Cahier des charges adapté au cadre pédagogique de l'IUT. — fin du document.